

PANDUAN LENGKAP CLASS DIAGRAM UML

Sistem Manajemen Masjid

Dokumen ini menjelaskan struktur class diagram UML untuk proyek Sistem Manajemen Masjid, mencakup setiap class, atribut, method, serta seluruh jenis relasi antar class.

BAGIAN 1 — APA ITU CLASS DIAGRAM?

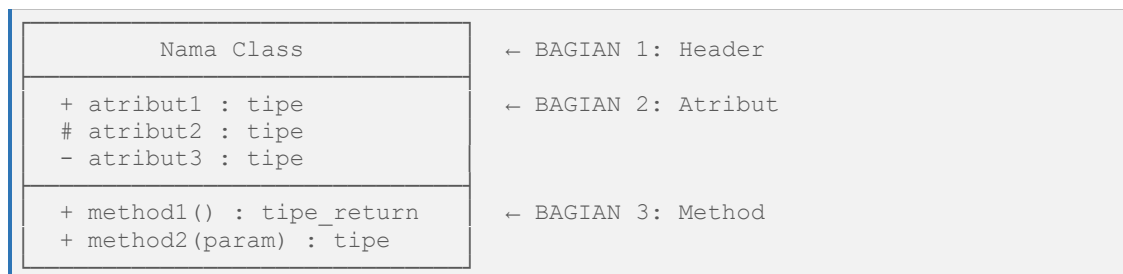
Class diagram adalah **peta visual** dari semua class yang ada di dalam project. Tujuannya adalah menunjukkan:

- Apa saja class yang dibuat
- Apa isi setiap class (atribut & method)
- Bagaimana hubungan antar class

Bayangkan class diagram seperti **denah rumah** — sebelum membangun rumah, arsitek membuat denah dulu. Class diagram adalah "denah" dari project Python kita.

BAGIAN 2 — BENTUK DASAR SEBUAH CLASS BOX

Setiap class digambarkan sebagai kotak persegi yang dibagi 3 bagian:



Tanda Visibility / Akses Modifier

Tanda	Nama	Artinya
+	Public	Bisa diakses dari mana saja
#	Protected	Hanya bisa diakses dari dalam class & class turunannya
-	Private	Hanya bisa diakses dari dalam class itu sendiri
*	Abstract	Method wajib di-override oleh class turunan

BAGIAN 3 — PENJELASAN SETIAP CLASS

Class 1: BaseModel

Lokasi file: `models/base_model.py`

«abstract» <i>BaseModel</i>	
- __id : str	← PRIVATE (tanda: -)
# _created_at : str	← PROTECTED (tanda: #)
— <i>Methods</i> —	
+ id : str	«property»
+ created_at : str	«property»
+ id(value)	«setter»
+ get_info() : dict	«method biasa»
+ __repr__() : str	
+ __eq__(other) : bool	
— <i>abstract</i> — - - - - -	
* to_dict() : dict	«abstract»
* from_dict(data) : BaseModel	«abstract»
* validate() : bool	«abstract»

Kenapa header ditulis «**abstract**» dan italic? Karena class ini tidak bisa dibuat objectnya langsung. Di Python ditandai dengan ABC. Di diagram UML, nama class abstract ditulis miring (italic) dan ada label «abstract» di atasnya.

Contoh Kode — models/base_model.py

```
1  from abc import ABC, abstractmethod
2
3  class BaseModel(ABC):          # ← ABC = Abstract Base Class
4
5      def __init__(self, id=None):
6          self.__id = id if id else str(uuid.uuid4())[0:8] # - __id : str (PRIVATE)
7          self._created_at = datetime.now().strftime(...) # # _created_at : str (PROTECTED)
8
9      @property
10     def id(self):                # + id : str «property»
11         return self.__id
12
13     @id.setter
14     def id(self, value):         # + id(value) «setter»
15         ...
16
17     def get_info(self):          # + get_info() : dict
18         return {...}
19
20     @abstractmethod
21     def to_dict(self):           # * to_dict()* : dict (ABSTRACT)
22         pass
23
24     @abstractmethod
25     def from_dict(cls, data):    # * from_dict(data)* : BaseModel (ABSTRACT)
26         pass
27
28     @abstractmethod
29     def validate(self):         # * validate()* : bool (ABSTRACT)
30         pass
```

Apa maksud ABSTRACT method? Method `to_dict`, `from_dict`, `validate` tidak punya isi (hanya `pass`). Artinya: "Siapun yang mewarisi `BaseModel`, WAJIB mengisi method ini". Kalau tidak diisi, Python akan error. Inilah konsep **Abstraction** dalam OOP.

Class 2: Artikel

Lokasi file: `models/artikel.py`

Artikel	
+ KATEGORI_VALID : list	← CLASS ATTRIBUTE (huruf kapital)
+ judul : str	← PUBLIC
+ konten : str	← PUBLIC
+ penulis : str	← PUBLIC
+ kategori : str	← PUBLIC
# _status : str	← PROTECTED
— Methods —	
+ __init__(...)	
+ status : str	«property»
+ status(value)	«setter»
+ to_dict() : dict	← Override dari BaseModel
+ from_dict(data) : Artikel	← Override dari BaseModel
+ validate() : bool	← Override dari BaseModel
+ publish() : str	
+ unpublish() : str	
+ get_preview(panjang : int) : str	

Contoh Kode — models/artikel.py

```
1  from models.base_model import BaseModel
2
3  class Artikel(BaseModel):          # ← Mewarisi BaseModel (panah inheritance)
4
5      KATEGORI_VALID = [...]        # + KATEGORI_VALID : list (CLASS ATTRIBUTE)
6
7      def __init__(self, judul, konten, penulis,
8                    kategori="Umum", status="draft",
9                    id=None, created_at=None):
10
11         super().__init__(id)       # ← Memanggil constructor BaseModel
12
13         self.judul    = judul      # + judul : str    (PUBLIC)
14         self.konten   = konten     # + konten : str   (PUBLIC)
15         self.penulis  = penulis    # + penulis : str  (PUBLIC)
16         self.kategori = kategori   # + kategori : str (PUBLIC)
17         self._status  = status     # # _status : str  (PROTECTED)
18
19     @property
20     def status(self):              # + status : str «property»
21         return self._status
22
23     @status.setter
24     def status(self, value):      # + status(value) «setter»
25         if value not in ["draft", "published"]:
26             raise ValueError(...)
27         self._status = value
28
29     def to_dict(self):            # + to_dict() : dict (OVERRIDE)
30         return {"id": self.id, "judul": self.judul, ...}
31
32     @classmethod
33     def from_dict(cls, data):     # + from_dict(data) : Artikel (OVERRIDE)
34         return cls(...)
35
36     def validate(self):           # + validate() : bool (OVERRIDE)
37         if not self.judul:
38             raise ValueError("Judul tidak boleh kosong!")
39         return True
40
41     def publish(self):            # + publish() : str
42         self._status = "published"
43         return f"Artikel '{self.judul}' dipublish!"
44
45     def get_preview(self, panjang=100): # + get_preview(panjang) : str
46         return self.konten[:panjang] + "..."
```

Class 3: Kajian

Lokasi file: `models/kajian.py`

Kajian	
+ HARI_VALID : list	← CLASS ATTRIBUTE
+ judul : str	← PUBLIC
+ materi : str	← PUBLIC
+ lokasi : str	← PUBLIC
+ hari : str	← PUBLIC
+ waktu : str	← PUBLIC
+ deskripsi : str	← PUBLIC
# _kuota : int	← PROTECTED, tipe int
— Methods —	
+ __init__(...)	
+ kuota : int	«property»
+ kuota(value)	«setter»
+ to_dict() : dict	← Override
+ from_dict(data) : Kajian	← Override
+ validate() : bool	← Override
+ get_jadwal_lengkap() : str	
+ tambah_kuota(jumlah : int) : str	

Contoh Kode — models/kajian.py

```
1  from models.base_model import BaseModel
2
3  class Kajian(BaseModel):
4
5      HARI_VALID = ["Senin","Selasa",...,"Ahad"] # + HARI_VALID : list
6
7      def __init__(self, judul, penerbit, lokasi,
8                  hari, waktu, deskripsi="",
9                  kuota=100, id=None, created_at=None):
10
11          super().__init__(id)
12
13          self.judul      = judul      # + judul : str
14          self.penerbit    = penerbit   # + penerbit : str
15          self.lokasi      = lokasi     # + lokasi : str
16          self.hari        = hari       # + hari : str
17          self.waktu       = waktu      # + waktu : str
18          self.deskripsi   = deskripsi  # + deskripsi : str
19          self._kuota      = kuota      # # _kuota : int (PROTECTED)
20
21      @property
22      def kuota(self):                  # + kuota : int «property»
23          return self._kuota
24
25      @kuota.setter
26      def kuota(self, value):           # + kuota(value) «setter»
27          if value <= 0:
28              raise ValueError("Kuota harus positif!")
29          self._kuota = value
30
31      def to_dict(self):                 # Override – isi BERBEDA dari Artikel
32          return {"id": self.id, "judul": self.judul,
33                  "penerbit": self.penerbit, ...}
34
35      def get_jadwal_lengkap(self):     # + get_jadwal_lengkap() : str
36          return f"{self.hari}, {self.waktu} WIB – {self.lokasi}"
37
38      def tambah_kuota(self, jumlah):   # + tambah_kuota(jumlah) : str
39          self._kuota += jumlah
40          return f"Kuota total: {self._kuota}"
```


Class 4: Pengumuman

Lokasi file: `models/pengumuman.py`

Pengumuman	
+ PRIORITAS_VALID : list	← CLASS ATTRIBUTE
+ TIPE_VALID : list	← CLASS ATTRIBUTE
+ judul : str	← PUBLIC
+ isi : str	← PUBLIC
+ tipe : str	← PUBLIC
+ tanggal_berlaku : str	← PUBLIC
- __prioritas : str	← PRIVATE (dua underscore)
# _aktif : bool	← PROTECTED
— Methods —	
+ __init__(...)	
+ prioritas : str	«property»
+ prioritas(value)	«setter»
+ aktif : bool	«property»
+ aktif(value)	«setter»
+ to_dict() : dict	← Override
+ from_dict(data) : Pengumuman	← Override
+ validate() : bool	← Override
+ aktifkan() : str	
+ nonaktifkan() : str	
+ get_badge() : str	

Contoh Kode — models/pengumuman.py

```
1 from models.base_model import BaseModel
2
3 class Pengumuman(BaseModel):
4
5     PRIORITAS_VALID = ["rendah", "normal", "tinggi"] # + PRIORITAS_VALID : list
6     TIPE_VALID = ["Umum", "Kegiatan", "Keuangan", "Darurat"] # + TIPE_VALID : list
7
8     def __init__(self, judul, isi, tipe="Umum",
9                 tanggal_berlaku="", prioritas="normal",
10                aktif=True, id=None, created_at=None):
11
12         super().__init__(id)
13
14         self.judul = judul # + judul : str (PUBLIC)
15         self.isi = isi # + isi : str (PUBLIC)
16         self.tipe = tipe # + tipe : str (PUBLIC)
17         self.tanggal_berlaku = tanggal_berlaku # + tanggal_berlaku : str
18
19         self.__prioritas = prioritas # - __prioritas : str (PRIVATE!)
20         self._aktif = aktif # # _aktif : bool (PROTECTED)
21
22     @property
23     def prioritas(self): # + prioritas : str «property»
24         return self.__prioritas # ← satu-satunya cara akses __prioritas
25
26     @prioritas.setter
27     def prioritas(self, value): # + prioritas(value) «setter»
28         if value not in self.PRIORITAS_VALID:
29             raise ValueError(f"Prioritas tidak valid!")
30         self.__prioritas = value
31
32     def aktifkan(self): # + aktifkan() : str
33         self._aktif = True
34         return f"Pengumuman '{self.judul}' diaktifkan!"
35
36     def get_badge(self): # + get_badge() : str
37         badge = {"rendah": "secondary", "normal": "primary", "tinggi": "danger"}
38         return badge.get(self.__prioritas, "primary")
```

Class 5: Masjid

Lokasi file: `models/masjid.py`

Masjid	
+ nama : str	← PUBLIC
+ alamat : str	← PUBLIC
+ kota : str	← PUBLIC
+ telepon : str	← PUBLIC
+ email : str	← PUBLIC
+ deskripsi : str	← PUBLIC
# _artikel : list	← PROTECTED (list of Artikel)
# _kajian : list	← PROTECTED (list of Kajian)
# _pengumuman : list	← PROTECTED (list of Pengumuman)
— Methods —	
+ __init__(nama, alamat, kota, ...)	
+ get_artikel(hanya_published) : list	
+ get_artikel_by_id(id) : Artikel	
+ tambah_artikel(artikel) : tuple	
+ edit_artikel(id, data) : tuple	
+ hapus_artikel(id) : tuple	
+ get_kajian() : list	
+ tambah_kajian(kajian) : tuple	
+ edit_kajian(id, data) : tuple	
+ hapus_kajian(id) : tuple	
+ get_pengumuman(hanya_aktif) : list	
+ tambah_pengumuman(p) : tuple	
+ edit_pengumuman(id, data) : tuple	
+ hapus_pengumuman(id) : tuple	
+ muat_semua_data() : Masjid	
+ get_statistik() : dict	
+ to_dict() : dict	
- _simpan_artikel() : void	← PRIVATE helper
- _simpan_kajian() : void	← PRIVATE helper
- _simpan_pengumuman() : void	← PRIVATE helper
- _tulis_json(path, data) : void	← PRIVATE helper
- _baca_json(path) : list	← PRIVATE helper

Contoh Kode — models/masjid.py

```
1 from models.artikel import Artikel
2 from models.kajian import Kajian
3 from models.pengumuman import Pengumuman
4
5 class Masjid:                                # ← TIDAK extends BaseModel
6
7     def __init__(self, nama, alamat, kota,
8                 telepon="", email="", deskripsi=""):
9
10         self.nama      = nama    # + nama : str   (PUBLIC)
11         self.alamat     = alamat  # + alamat : str
12         self.kota       = kota    # + kota : str
13         self.telepon    = telepon
14         self.email      = email
15         self.deskripsi  = deskripsi
16
17         # OBJECT RELATIONSHIP – Masjid has-a Artikel/Kajian/Pengumuman
18         self._artikel   = []      # # _artikel : list (PROTECTED)
19         self._kajian    = []      # # _kajian : list
20         self._pengumuman = []     # # _pengumuman : list
21
22     def tambah_artikel(self, artikel: Artikel): # parameter tipe Artikel
23         try:
24             artikel.validate()
25             self._artikel.append(artikel) # ← simpan object Artikel ke list
26             self._simpan_artikel()        # ← panggil method private
27             return True, "Berhasil!"
28         except ValueError as e:
29             return False, str(e)
30
31     def _simpan_artikel(self):                # - _simpan_artikel() : void (PRIVATE)
32         self._tulis_json(
33             Config.ARTIKEL_FILE,
34             [a.to_dict() for a in self._artikel]
35         )
36
37     def _tulis_json(self, filepath, data):    # - _tulis_json : void (PRIVATE)
38         with open(filepath, "w") as f:
39             json.dump(data, f)
40
41     def _baca_json(self, filepath):           # - _baca_json : list (PRIVATE)
42         with open(filepath, "r") as f:
43             return json.load(f)
44
45     def get_statistik(self):                  # + get_statistik() : dict
46         return {
47             "total_artikel": len(self._artikel),
48             "total_kajian" : len(self._kajian),
49             ...
50         }
```

Class 6: Config

Lokasi file: `config.py` (di root project, bukan di `models/`)

Config	
+ APP_NAME : str	← Semua PUBLIC dan HURUF KAPITAL
+ APP_VERSION : str	← Class Attribute (tanpa <code>__init__</code>)
+ DEBUG : bool	
+ SECRET_KEY : str	
+ BASE_DIR : str	
+ DATA_DIR : str	
+ ARTIKEL_FILE : str	
+ KAJIAN_FILE : str	
+ PENGUMUMAN_FILE : str	
+ MASJID_FILE : str	
+ ADMIN_USERNAME : str	
+ ADMIN_PASSWORD : str	
+ ITEMS_PER_PAGE : int	
— Methods —	

Catatan: Tidak ada bagian method karena Config hanya menyimpan nilai konstanta konfigurasi.

Contoh Kode — `config.py`

```
1 import os
2
3 class Config:                                # ← class biasa, tidak extends apapun
4
5     APP_NAME = "Sistem Manajemen Masjid"    # + APP_NAME : str
6     APP_VERSION = "1.0.0"                   # + APP_VERSION : str
7     DEBUG = True                            # + DEBUG : bool
8     SECRET_KEY = "masjid-secret-key-2026"   # + SECRET_KEY : str
9
10    BASE_DIR = os.path.dirname(os.path.abspath(__file__))
11    DATA_DIR = os.path.join(BASE_DIR, "data")
12
13    ARTIKEL_FILE = os.path.join(DATA_DIR, "artikel.json") # + ARTIKEL_FILE : str
14    KAJIAN_FILE = os.path.join(DATA_DIR, "kajian.json")
15    PENGUMUMAN_FILE = os.path.join(DATA_DIR, "pengumuman.json")
16    MASJID_FILE = os.path.join(DATA_DIR, "masjid.json")
17
18    ADMIN_USERNAME = "admin"                  # + ADMIN_USERNAME : str
19    ADMIN_PASSWORD = "masjid2026"            # + ADMIN_PASSWORD : str
20    ITEMS_PER_PAGE = 6                       # + ITEMS_PER_PAGE : int
```

Class 7: UserMixin (Library Eksternal)

Sumber: **Library Flask-Login** — bukan file yang kita buat sendiri. Diinstall via: `pip install flask-login`

«interface» UserMixin
— Methods —
+ <code>is_authenticated()</code> : bool
+ <code>is_active()</code> : bool
+ <code>is_anonymous()</code> : bool
+ <code>get_id()</code> : str

Kenapa garis putus-putus? Karena ini bukan class yang kita buat sendiri — ini dari library eksternal. Di `draw.io`, set `strokeDashArray` menjadi putus-putus untuk membedakannya dari class milik kita.

Class 8: AdminUser

Lokasi file: `app.py` (di root project, bukan di `models/`)

AdminUser
+ <code>id</code> : str ← PUBLIC
+ <code>username</code> : str ← PUBLIC
— Methods —
+ <code>__init__(id: str, username: str)</code>
+ <code>__repr__()</code> : str
+ <code>is_authenticated()</code> : bool ← Inherited dari UserMixin
+ <code>is_active()</code> : bool ← Inherited dari UserMixin
+ <code>is_anonymous()</code> : bool ← Inherited dari UserMixin
+ <code>get_id()</code> : str ← Inherited dari UserMixin

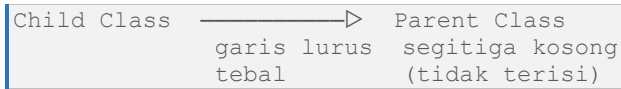
Contoh Kode — `app.py`

```
1 from flask_login import UserMixin, LoginManager
2
3 class AdminUser(UserMixin):    # ← Mewarisi UserMixin (panah inheritance)
4
5     def __init__(self, id, username):
6         self.id = id          # + id : str (PUBLIC)
7         self.username = username # + username : str (PUBLIC)
8
9     def __repr__(self):        # + __repr__() : str
10        return f"<AdminUser {self.username}>"
11
12        # is_authenticated(), is_active(), get_id()
13        # TIDAK perlu ditulis ulang — otomatis diwarisi dari UserMixin
```

BAGIAN 4 — PENJELASAN LENGKAP SEMUA GARIS PANAH

Panah 1: Inheritance (Pewarisan / Is-A)

Bentuk visual:



Di diagram terdapat 4 panah inheritance:

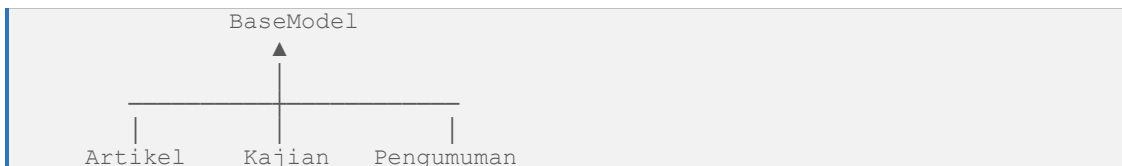
Panah	Dari	Ke	Bukti di Kode
1	Artikel	BaseModel	<code>class Artikel(BaseModel):</code>
2	Kajian	BaseModel	<code>class Kajian(BaseModel):</code>
3	Pengumuman	BaseModel	<code>class Pengumuman(BaseModel):</code>
4	AdminUser	UserMixin	<code>class AdminUser(UserMixin):</code>

Cara membaca panah: Panah dari Artikel ke BaseModel dibaca: *"Artikel adalah sebuah BaseModel"*

Aturan arah panah: BENAR: `Artikel —> BaseModel` (child menunjuk ke parent)

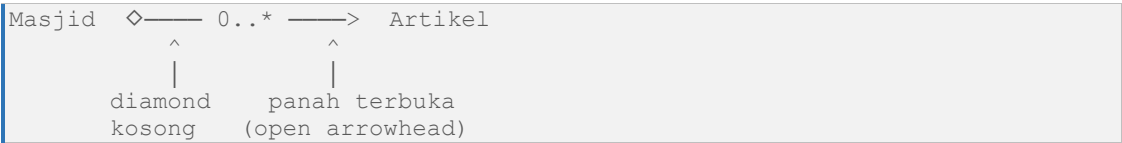
SALAH: `BaseModel —> Artikel` (kebalik!)

Cara menggambar 3 panah ke 1 parent di diagram:



Panah 2: Aggregation (Has-A Relationship)

Bentuk visual:



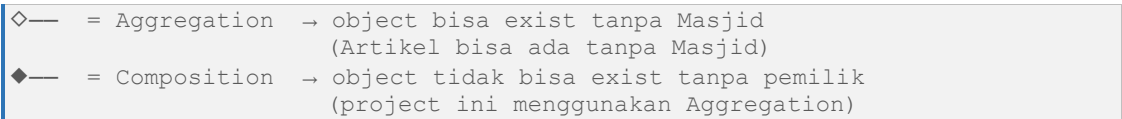
Di diagram terdapat 3 panah aggregation:

Panah	Dari (Pemilik)	Ke (Dimiliki)	Multiplicity	Bukti di Kode
1	Masjid ◇	Artikel	0..*	self._artikel = []
2	Masjid ◇	Kajian	0..*	self._kajian = []
3	Masjid ◇	Pengumuman	0..*	self._pengumuman = []

Arti Label 0..*

- 0 = minimum nol (boleh kosong, tidak ada data)
- .. = sampai
- * = tak terbatas (bisa banyak sekali)
- 0..* = nol sampai tak terbatas

Diamond Kosong vs Diamond Terisi



Perbandingan Inheritance vs Aggregation

Aspek	Inheritance	Aggregation
Simbol	—▷ segitiga kosong	◇— diamond kosong
Hubungan	is-a	has-a
Contoh kode	class Artikel(BaseModel)	self._artikel = []
Dibaca	"Artikel adalah BaseModel"	"Masjid punya Artikel"
Diamond	tidak ada	di sisi yang memiliki

Panah 3: Dependency (Uses)

Bentuk visual:

```
Config - - - - - > Masjid
      garis putus-putus panah terbuka
Label di panah: «uses»
```

Di diagram terdapat 2 panah dependency:

Panah	Dari	Ke	Lokasi di Kode
1	Config ---«uses»-->	Masjid	masjid.py — Config.ARTIKEL_FILE
2	Config ---«uses»-->	AdminUser	routes/admin.py — Config.ADMIN_USERNAME

Perbandingan Semua Jenis Panah

Aspek	Dependency	Inheritance	Aggregation
Simbol	--> putus	—▷ tebal	◇— diamond
Arti	menggunakan	adalah	memiliki
Contoh	from config import Config	class A(B):	self.list = []
Di kode	import / from	class A(B):	atribut list

BAGIAN 5 — RINGKASAN PETA OOP PROJECT

Konsep OOP	Class	File	Bukti di Kode
Class	Semua	Semua file models/	class NamaClass:
Object	Masjid	masjid.py (baris akhir)	masjid = Masjid(...)
Attribute	Semua	Semua __init__	self.judul = judul
Method	Semua	Semua file models/	def nama_method(self):
Public (+)	Semua	Semua	self.judul, self.nama
Protected (#)	BaseModel, Artikel, Kajian, Pengumuman	models/	self._status, self._kuota
Private (-)	BaseModel, Pengumuman	base_model.py, pengumuman.py	self.__id, self.__prioritas
Constructor	Semua	Semua	def __init__(self, ...)
Inheritance	Artikel/Kajian/Pengumuman	artikel.py, kajian.py, pengumuman.py	class Artikel(BaseModel)
Polymorphism	Artikel/Kajian/Pengumuman	models/	to_dict() isi berbeda di tiap class
Abstraction	BaseModel	base_model.py	ABC + @abstractmethod
Encapsulation	Semua	models/	@property + @setter
Object Relationship	Masjid	masjid.py	self._artikel = []
Dynamic Behavior	Artikel, Pengumuman, Masjid	models/	publish(), aktifkan(), get_statistik()
Exception Handling	Semua	models/	try: ... except ValueError:
ValueError	Semua	validate() di tiap class	raise ValueError("...")

Dengan penjelasan dalam dokumen ini, Anda seharusnya sudah bisa membuat diagram yang 100% akurat — mulai dari bentuk box, isi atribut & method, tanda akses modifier, sampai semua jenis panah beserta artinya. Setiap bagian diagram sudah dikaitkan langsung ke kode nyata di project!